CS 5045: Computation for the Data Sciences

Practice Midterm Exam — Complete Answer Key & Explanations

Part 1: Code Trace (24 Points)

Q1. Code Output:

```
scores = [65, 80, 45, 90, 72]
total = 0
for s in scores:
    if s >= 70:
        total += 10
print(total)
```

Output:
30

*Explanation: The loop checks each score. Three scores (80, 90, 72) meet the condition >= 70. Each valid score adds 10 to total, so 3 * 10 = 30.*

Q2. Code Output:

```
def check_speed(speed):
    if speed > 70:
        return "Ticket"
    elif speed > 55:
        return "Warning"
    else:
        return "Safe"

print(check_speed(60))
print(check_speed(50))
```

Output:
Warning
Safe

Explanation: For check_speed(60), speed > 70 is False, but speed > 55 is True, so it hits the elif block and returns 'Warning'. For check_speed(50), both conditions fail, falling through to the else block which returns 'Safe'.

Q3. Code Output:

```
item = {"name": "Laptop", "price": 999.99, "in_stock": True}
print(item["price"] < 500)
print(type(item["in_stock"]))
```

Output:
False
<class 'bool'>

Explanation: item['price'] is 999.99, which is not less than 500 (False). item['in_stock'] holds the value True, which belongs to the primitive boolean type.

Q4. Code Output:

```
user = {"username": "data_sci", "level": 2}
user["level"] = 3
user["is_admin"] = False
print(user["level"])
print(len(user))
```

Output:
3
3

Explanation: Assigning to an existing key ('level') replaces its value from 2 to 3. Adding a new key ('is_admin') increases the total count of key-value pairs in the dictionary to 3.

Q5. Code Output:

```
def calc_bmi(weight_kg, height_m=1.75):
    return round(weight_kg / (height_m ** 2), 1)

print(calc_bmi(70.0))
print(calc_bmi(70.0, height_m=2.0))
```

Output:
22.9
17.5

Explanation: The first call relies on the default parameter value height_m=1.75 (70.0 / 1.75^2 = 22.857..., rounded to 22.9). The second call overrides the parameter with height_m=2.0 (70.0 / 2.0^2 = 17.5).

Q6. Code Output:

```
def compute_square(n):
    return n * n

def display_square(n):
    print(n * n)

x = compute_square(4)
y = display_square(4)
print(x)
print(y)
```

Output:
16
16
None

Explanation: When y = display_square(4) runs, it prints 16 directly to the screen. Because display_square lacks a return statement, it implicitly returns None, meaning y evaluates to None. Finally, print(x) prints 16 and print(y) prints None.

Q7. Code Output:

```
class Device:
    def __init__(self, brand, battery_pct):
        self.brand = brand
        self.battery_pct = battery_pct

    def needs_charge(self):
        return self.battery_pct < 20

d1 = Device("Apple", 15.5)
d2 = Device("Samsung", 85.0)
print(d1.needs_charge())
print(d2.needs_charge())
print(d1.brand)
```

Output:
True
False
Apple

Explanation: d1 has a battery of 15.5, which is < 20 (True). d2 has 85.0, which is not < 20 (False). d1.brand accesses the string literal stored during initialization.

Q8. Code Output:

```
class Plan:
    def __init__(self, client, base_rate):
        self.client = client
        self.base_rate = base_rate

    def final_cost(self):
        return self.base_rate

class PremiumPlan(Plan):
    def __init__(self, client, base_rate, concierge_fee):
        super().__init__(client, base_rate)
        self.concierge_fee = concierge_fee

    def final_cost(self):
        return self.base_rate + self.concierge_fee

p1 = Plan("John", 100)
p2 = PremiumPlan("Sarah", 100, 50)
print(p1.final_cost())
print(p2.final_cost())
print(p2.client, p2.concierge_fee)
```

Output:
100
150
Sarah 50

Explanation: p1 uses Plan's final_cost method (100). p2 invokes the overridden method in PremiumPlan, computing base_rate + concierge_fee (100 + 50 = 150). super().__init__ preserves fields on self.

Part 2: Types and Representation (24 Points)

Q9. Primitive Types Identification:
• False → **bool**
• "True" → **str** (Any value wrapped in quotes becomes a string)
• 42 → **int** (Whole numerical integer)
• 42.0 → **float** (Numbers written with an explicit decimal points)
• "-1" → **str** (A negative integer representation wrapped inside literal text quotes)

Q10. The Four-Line Audit Framework:
1. df.shape — *Answers: Did the complete dataset load properly? Check row and column counts.*
2. df.dtypes — *Answers: Did the columns load under the correct data types? Watch for numeric fields loaded as text.*
3. df.head() — *Answers: Does the structural framing look correct visually? Spot alignment or truncation errors.*
4. df.isna().sum() — *Answers: Where are the gaps and structural omissions in our records? Count null values per column.*

Q11. Silent Type Conversion Failure:
Assigned Type: object or str
Explanation: Because the text string contains a non-numeric symbol character ('$'), pandas loads the complete collection as text strings rather than real numbers. Computational math operations like calculating the sample average (.mean()), summary totals (.sum()), or comparative filtering calculations (e.g., > 50) will instantly fail or trigger type exceptions because mathematical operators are undefined on raw string patterns.

Q12. csv.DictReader Type Normalization:

```
# Extracting/Converting price safely:
converted_price = int(row["price"])

# Extracting/Converting tax_exempt safely:
is_exempt = row["tax_exempt"] == "True"
```

Explanation: Fields extracted via csv.DictReader are invariably primitive text strings. Running bool('False') evaluates to True because Python checks string containment size. Explicit conditional comparison fixes this behavior safely.

Q13. Boolean Verification Mechanics (.sum() vs .mean()):
• .sum() treats True elements as 1 and False as 0, thereby counting the absolute total number of customers who downgraded their status.
• .mean() computes the literal proportion or percentage share of total accounts who downgraded relative to the complete population base.

Part 3: The Five Operations (8 Points)

Q14. df = pd.read_csv('hospital_records.csv')

Correct Framework Classification: Represent

Reasoning: Loading exterior datasets from text-based storage disk arrays straight into live active volatile computer memory frames.

Q15. Classifying metric ranges into tier levels (without removing structural records).

Correct Framework Classification: Transform

Reasoning: Changing categorical states or computing new metrics while leaving the baseline row count unchanged.

Q16. df.groupby('department')['salary'].mean()

Correct Framework Classification: Aggregate

Reasoning: Collapsing large collections of row items into centralized, segmented summary groups.

Q17. Isolating all structural records matching age >= 65.

Correct Framework Classification: Filter

Reasoning: Pruning away records that fail validation constraints, resulting in fewer rows.

Q18. Drafting text summaries or metric visualizations to guide executive choices.

Correct Framework Classification: Communicate

Reasoning: Formatting and explaining data results to humans without changing the structural source records.

Part 4: Write Code by Hand (25 Points)

Q19. For Loop Conditional Filtering Block:

```
for sal in salaries:
    if sal > 100000:
        print(sal)
```

Q20. In-Place Dictionary Modification Loop:

```
for person in staff:
    if person["sales"] >= 10000 and person["years"] > 2:
        person["bonus"] = 1000
    elif person["sales"] < 10000:
        person["bonus"] = 0
    else:
        person["bonus"] = 500
```

Q21. Categorical Accumulation and Group Aggregation:

```
# Grouping logic implementation:
by_cat = {}
for row in transactions:
    cat = row["category"]
    if cat not in by_cat:
        by_cat[cat] = []
    by_cat[cat].append(row["amount"])

# Averaging calculations:
for cat, amounts in by_cat.items():
    avg = sum(amounts) / len(amounts)
    print(cat, avg)
```

Q22. Class Inheritance and Method Overriding Execution:

```
class Manager(Employee):
    def __init__(self, name, base_salary, bonus):
        super().__init__(name, base_salary)
        self.bonus = bonus

    def calculate_pay(self):
        return self.base_salary + self.bonus
```

Part 5: Short Answer (19 Points)

Q23. Object Instance Context Isolation Mechanics:
Every distinct instance object stores its unique state attributes within its own namespace. When an instance calls a shared class method, Python implicitly forwards a reference to that specific instance as the first structural argument, mapped to the parameter self. Because the method evaluates fields using self.attribute_name, the unique values belonging to the calling instance are read, producing distinct, localized outcomes from identical code instructions.

Q24. Proxy Variables and Latent Informational Redundancy:
A proxy variable is an explicitly tracking descriptive field that correlates heavily with a sensitive or legally protected characteristic due to systemic patterns, historical inequities, or real-world biases (such as using geographic zip codes that mimic historic redlining racial segregations). Simply removing the core column (e.g., race or gender) leaves the predictive model completely uncorrected, because the proxy variables carry the identical historical informational signal, causing the model to learn the same biased historical choices.

Q25. Metric Workflow Chronological Pipeline Steps:
1. **Represent:** Load the complete primary dataset containing worker data from its raw physical file layer directly into local computer program arrays.
2. **Filter (Optional):** Safely eliminate corrupted rows or missing entries containing null values in sick day records.
3. **Aggregate:** Group the data by worker location status (remote vs. in-office) and calculate the mathematical average or mean number of sick days taken for each cluster.

4. **Communicate:** Render the resulting means within an explanatory text block summary table to clearly present the conclusions to the manager.